PRESENTLY CAMPUS

A Proxy-Resistant QR Attendance and Analytics System

A Project Report
submitted for the course
IOTBDM (Semester VII)

Submitted by

Abhishek Kokadwar	(2023IMG-002)
Nabaskar Brahma	(2023IMG-035)



ABV-INDIAN INSTITUTE OF INFORMATION TECHNOLOGY AND MANAGEMENT GWALIOR

GWALIOR, INDIA

NOVEMBER, 2026

Abstract

Our project proposal presented Presently, a phone app in which students marked their own classes and saw, per subject, how many more they could miss before falling below 75%. Its figures were only as good as the students' taps, and it planned three next steps: automatic capture in the classroom, class-wide analysis for faculty, and college rules with a cutoff date. Taking attendance automatically raises its own problem: a fixed QR code can be photographed and shared, so proxies continue online.

This report presents **Presently Campus**, a web application that records class attendance by QR scan while making proxy attendance impractical. The teacher projects a code that changes every ten seconds and is signed with a secret only the server shares. Each student's account is bound to one phone through a cryptographic key that cannot leave the phone's browser, and every scan is signed with it. A server function checks each scan in nine steps, from code expiry to one mark per phone per class, and records every attempt. Students see their attendance in each subject together with how many classes they can still miss or must attend to reach 75%; teachers get a live count, a roster, corrections with reasons, and analysis of who is heading below the target.

The system is built with React and TypeScript as an installable web app on Supabase (PostgreSQL with Row Level Security, Edge Functions and Realtime). The system is complete and deployed for use with college email addresses, and passes 184 automated tests, six database test suites and ten acceptance tests.

Keywords: *attendance system, QR code, proxy prevention, device binding, HMAC, ECDSA, Progressive Web App, PostgreSQL, Row Level Security*

Contents

1. Introduction **1**
1.1 Background 1
1.2 Problem statement 1
1.3 Objectives 1
2. Existing Methods and Their Limitations **1**
3. Requirements **2**
3.1 Users and roles 2
3.2 Functional requirements 2
3.3 Non-functional requirements 2
4. System Design **3**
4.1 Architecture 3
4.2 Technology stack 3
5. Anti-Proxy Design **4**
5.1 Rotating QR token 4
5.2 Device binding: one phone per account 4
5.3 Scan verification 5
5.4 Soft signals 5
6. Database Design **5**
7. Implementation **6**
7.1 Teacher's application 6
7.2 Analysis and data 7
7.3 Student's application 7
7.4 Development practice 8
8. Testing **8**
8.1 Automated tests 8
8.2 Acceptance tests 8
9. Results and Discussion **9**
9.1 Proposal against delivery 9
9.2 Outcome 9
9.3 Limitations 9
10. Future Scope **10**
11. Conclusion **10**
References **11**

1. Introduction

1.1 Background

Attendance decides whether a student may sit the end-semester examination, yet a student usually sees their official figure only when the college publishes it, close to the eligibility check. By then a subject may have slipped below 75% weeks earlier, and an overall 78% can hide one subject at 62%, since each subject is judged on its own.

Our project proposal (21 October 2026) presented **Presently**, a phone app in which a student marked each class themselves and saw, for every subject, how many classes they could still miss or must attend to get back to 75%. The proposal named its main limit: marking was manual, so the figures were only as good as the student's taps. It set three next steps: **automatic capture** of presence in the classroom by QR code or Bluetooth beacon, **class-wide analysis** for faculty, and **college rules with a cutoff date**.

This report presents **Presently Campus**, which delivers those steps. Attendance is now taken by the teacher by QR scan, and students no longer mark themselves: a mark exists only if the server accepted a valid scan or a teacher entered it by hand. This raises a new problem. A single fixed QR code is quick, but a photo of it sent to a WhatsApp group lets anyone mark themselves present from their hostel room, just as a friend can answer to an absent student's name in a roll call. Preventing such proxies became the central design problem of the project.

1.2 Problem statement

Build a system that takes the attendance of a full class in under three minutes without a roll call, makes proxy attendance impractical, and shows every student and teacher an accurate, subject-wise picture of attendance at any time, while keeping the data complete enough to analyse.

1.3 Objectives

- Let a teacher take attendance for a class of 60 in under three minutes, with no roll call.
- Make proxy attendance impractical: a forwarded photo of the code, someone else's account on a friend's phone, and one phone marking two students must all fail.
- Show each student their standing per subject: the percentage, and how many classes they can still miss or must attend to reach the 75% requirement.
- Give teachers a roster, the ability to correct marks with a reason, and analysis of who is heading below the target.
- Keep every scan attempt, accepted or rejected, as queryable data for the analysis part of the IoTBDM course.

Scope: one deployment serves one college, and sign-up is limited to its email domain (`iiitm.ac.in`). The app installs on a phone from the browser (a Progressive Web App), so no app-store release is needed. ERP integration and face recognition are out of scope.

2. Existing Methods and Their Limitations

Table 1 compares the common methods with this project. Two other ideas were rejected after study: browsers cannot read a phone's **MAC address**, and **one mark per IP address** would block genuine students, since a whole class on campus Wi-Fi shares one public address. The IP address is used only as a soft signal (Section 5.4).

Table 1: *Comparison of common attendance methods*

Method	Speed for 60 students	Resistance to proxies	Main limitation
Roll call	5 to 10 minutes	Low	Friends answer for absent students; wastes lecture time
Signed sheet	Passes around the class	Low	Signatures are copied; data must be typed up later
Biometric device	A queue at one device	High	Costly hardware, queues, and biometric data to protect
Static QR code	Under 2 minutes	Very low	A photo of the code works from anywhere
Presently Campus	Under 2 minutes	High	Needs a smartphone; teacher marks the rest by hand

3. Requirements

3.1 Users and roles

- **Student:** joins courses with a code, registers one phone, scans to mark attendance and sees their own figures.
- **Teacher:** creates courses, runs live sessions, sees and corrects attendance in their courses, approves phone changes, exports and analyses. Teachers are added only by an administrator, never by signing up.
- **Developer:** the two project members, who can switch their own account between the teacher and student views to test both sides.

3.2 Functional requirements

1. Sign in with a college email address, by a one-time link or a password.
2. Teachers create courses (name, code, section, term, target percentage) and share a six-character join code or QR code.
3. A teacher starts a session and projects a QR code that changes every ten seconds, with a live count of students marked.
4. A student scans the code from their registered phone and sees the result within two seconds: marked present, or exactly why not.
5. Closing a session makes every enrolled student who did not scan absent for it.
6. Teachers see a roster per course and correct any mark with a reason; every correction is logged.
7. Teachers review flagged scans, approve new phones, export attendance as CSV, and view analysis.

3.3 Non-functional requirements

- **Security:** every rule that decides whether a mark is valid runs on the server; the browser is never trusted.
- **Reliability:** the projected code keeps rotating even on a poor classroom connection.
- **Usability:** large text, labelled buttons and three text sizes, so that teachers who are not used to phone apps can use it comfortably; light and dark themes; works on phones and desktops.
- **Privacy:** store only what the checks need, and clear network addresses after 90 days, in line with India's Digital Personal Data Protection Act, 2023.

4. System Design

4.1 Architecture

The system has two clients, built as one React application that shows different screens by role, and one backend on Supabase, a hosted platform built on PostgreSQL. Figure 1 shows the parts and how a scan travels through them.

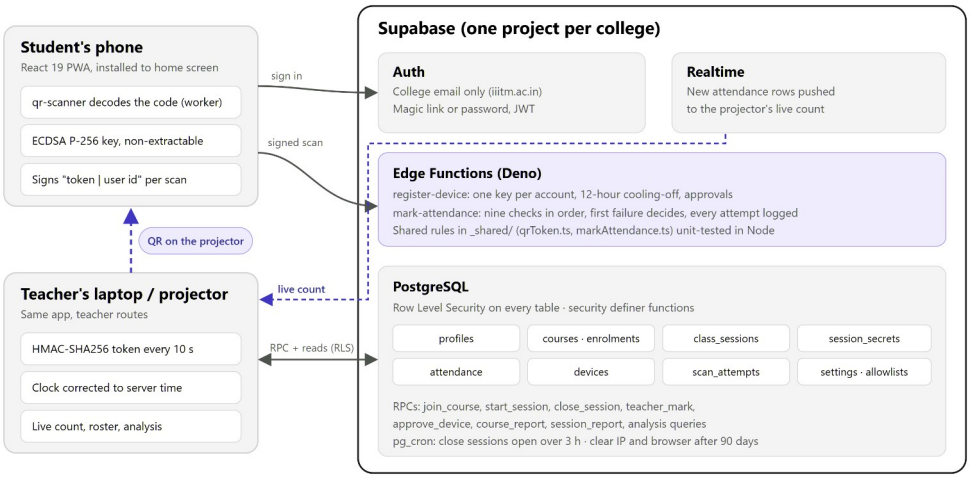


Figure 1: *System architecture of Presently Campus*

Most of the logic lives in the database. **Row Level Security** (RLS) policies, written next to each table, decide which rows each user may read. Students and teachers have no direct write access to attendance, sessions or devices at all: every write goes through a database function that checks the caller, or through one of two **Edge Functions**, small server programs that hold the scan and phone-registration rules. **Realtime** pushes each new attendance row to the teacher's projector so the live count moves as students scan, and **pg_cron** runs two scheduled jobs.

4.2 Technology stack

Table 2: *Technology stack*

Layer	Technology	Why it was chosen
Front end	React 19, TypeScript, Vite 7, Tailwind CSS 4	Typed components, fast builds, one design-token file for both themes
Installable app	vite-plugin-pwa (service worker, manifest)	Installs to the home screen and opens offline for reading
QR	qr-scanner (decoding), qrcode (drawing)	qr-scanner runs in a worker and works on iPhone Safari
Cryptography	Web Crypto API: HMAC-SHA256, ECDSA P-256	Built into every browser and into Deno; no extra library
Database	PostgreSQL with RLS, pg_cron	Access rules sit next to the data; reports computed in SQL
Server code	Supabase Edge Functions (Deno)	The two places that need server code: scanning and phone registration
Auth and live data	Supabase Auth, Supabase Realtime	Email sign-in with domain checks; live count without polling
Hosting	Vercel (web), Supabase (backend)	Deploys from the main branch on every merge

5. Anti-Proxy Design

This is the core of the project. Table 3 lists each way a student might try to mark someone else present, and what stops it.

Table 3: *Threats and countermeasures*

Attempt	What stops it
Photo or screenshot of the code sent to someone outside	The code changes every 10 seconds and expires within 20 (Section 5.1)
Absent student's account signed in on a friend's phone	Each account scans only from its one registered phone (Section 5.2)
One phone marking two students	One mark per phone per session, enforced by a unique constraint in the database
Clearing the app's data to register the phone for a second account	New phones cool off for 12 hours and can never mark a class that began before they were registered
Calling the API directly to insert attendance	Students have no write rights on attendance; only the server function writes
Replaying a captured scan after class	The session must be open, and the token carries its session and time window
A live video call of the code, or the absent student's phone carried in by a friend	Not blocked outright. Flagged for the teacher (Section 5.4); passkeys are future scope

5.1 Rotating QR token

When a teacher starts a session, the database creates a random 32-byte secret that only that teacher and the server can read. Time is divided into ten-second windows, and the projector derives the code for the current window from the secret:

```
window = floor(server_time_in_seconds / 10)
sig    = base64url( first 16 bytes of HMAC-SHA256(secret, "<session_id>.<window>") )
token  = "p1.<session_id>.<window>.<sig>"
```

Because the teacher's browser computes each code itself, much like an authenticator app, the code keeps rotating even if the classroom connection drops. When the session starts, the server also returns its current time, and the projector corrects its clock by the difference, so a laptop whose clock is two minutes wrong still produces valid codes. The server accepts the current and the previous window, so a code is valid for 10 to 20 seconds: long enough to scan, too short to forward. One module, `qrToken.ts`, is shared by the browser and the server, so signing and checking can never disagree.

5.2 Device binding: one phone per account

On a student's first sign-in, the app creates an **ECDSA P-256 key pair** with the browser's Web Crypto API and marks the private key **non-extractable**: it cannot be read or copied out of the browser, even by the app's own code. The key is kept in IndexedDB with persistent storage requested, and only the public key is sent to the server. Every scan is then signed with this private key, so a request that does not come from the registered phone fails. The server enforces these rules:

- A public key belongs to one account only; registering a key already used by another account fails.
- A student has at most one active phone. A first phone becomes usable after a 12-hour cooling-off period, or at once if a teacher approves it.
- A phone registered after a session began can never mark that session. This blocks the trick of clearing the app's data, signing in as an absent friend and registering the phone for them mid-class.

- A new phone for a student who already has one waits for a teacher's approval, which retires the old phone. Every step is logged.

5.3 Scan verification

A scan is one HTTPS request to the `mark-attendance` function carrying the token, the phone's identifier and the signature of `"<token>|<user_id>"`. The user is taken from the sign-in token, never from the request body. The function runs nine checks in order and stops at the first failure, so the student always learns the exact reason (Table 4). Every attempt, accepted or rejected, is written to `scan_attempts` with its reason, IP address and browser.

Table 4: *The nine checks of a scan, in order*

#	Check	Message shown on failure
1	Token parses and the session exists	That isn't a Presently code.
2	Session is open	This class has closed.
3	Signature valid for the current or previous window	Code expired. Scan the screen again.
4	Student is enrolled in the course	You're not in this course. Join it first.
5	Phone belongs to this student and is active	This phone isn't registered to you.
6	Cooling-off over and phone registered before the session began	New phones can't mark this class. Ask your teacher.
7	Request signature verifies with the phone's public key	This phone couldn't be verified.
8	Phone has not marked anyone else in this session	This phone has already marked someone in this class.
9	Student not already marked	Success either way: "Already marked present."

5.4 Soft signals

Some signals suggest a proxy but also have innocent causes, so they never reject a scan. They are stored as flags on the mark, and the teacher decides whether to keep or remove it:

- **Off campus:** the request came from outside the college's configured network ranges.
- **Shared connection:** more than three students scanned from the same address and browser within a few seconds.
- **Late:** the scan came more than a set number of minutes (10 by default) after the session started.

6. Database Design

The schema is built in seven migrations, one per phase of the build. Every table has Row Level Security turned on. Table 5 lists the main tables.

Table 5: *Main tables*

Table	Purpose
profiles	Name, roll number and role of every user; the role is set by triggers from the teacher list, never by the user
courses, enrolments	Courses with their join codes and targets, and which students have joined them
class_sessions	Each class held: open, closed or cancelled; at most one open per course
session_secrets	Each session's 32-byte secret, in a separate table that students cannot read at all
attendance	One row per present or excused mark, with method (QR or manual), phone, reason and flags
attendance_changes	Every manual correction, with what it replaced and who made it
devices, device_events	Each phone's public key and status, and a log of every registration, approval and revocation

Table	Purpose
scan_attempts	Every scan attempt, accepted or rejected, with the reason, IP address and browser
settings, allowlists	Allowed email domains, campus network ranges, the teacher list and the developer list

Absence is computed, not stored. A student is absent from a session if it is closed, they had joined the course before it began, and they have no attendance row. Closing a session therefore needs no extra writes, and a student who joins late is never counted absent for classes held before they joined. For each student and course, *held* is the closed sessions they were enrolled for minus excused ones, and *attended* is their present marks. With target t percent, the margins shown to students are:

$$\textit{classes that can still be missed} = \textit{floor}((100 \times \textit{attended} - t \times \textit{held}) / t)$$

$$\textit{classes that must be attended next} = \textit{ceil}((t \times \textit{held} - 100 \times \textit{attended}) / (100 - t))$$

These are the same formulas as in the proposal's Presently app, and its worked example still holds: 18 of 21 classes attended is 85.7%, and missing the next 3 gives 18 of 24, exactly 75%, so the student sees **3 spare**; a subject at 10 of 16 (62.5%) shows **attend next 8**. The margin assumes every class from now on is missed, so it is always true today, and attending only raises it.

These figures are computed by one SQL function, `course_figures`, used by the roster, the course overview, the student's own view and the CSV export, so no two screens can ever disagree.

7. Implementation

7.1 Teacher's application

A teacher lands on a dashboard: totals for students, classes held, average attendance and students below target, a bar for each course against its target, any live session, and the courses that need attention. Opening a course shows the roster as a register, sortable by roll number, name or percentage, with each student's status in words (Figure 2).

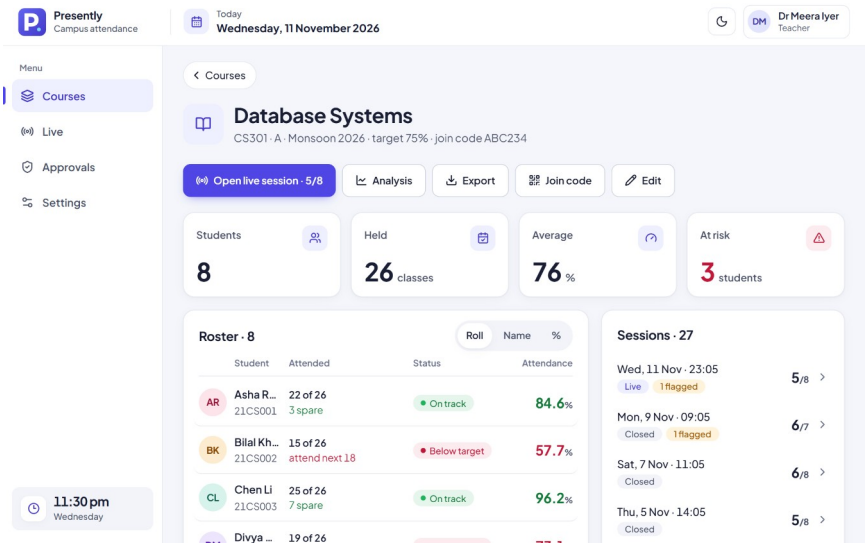
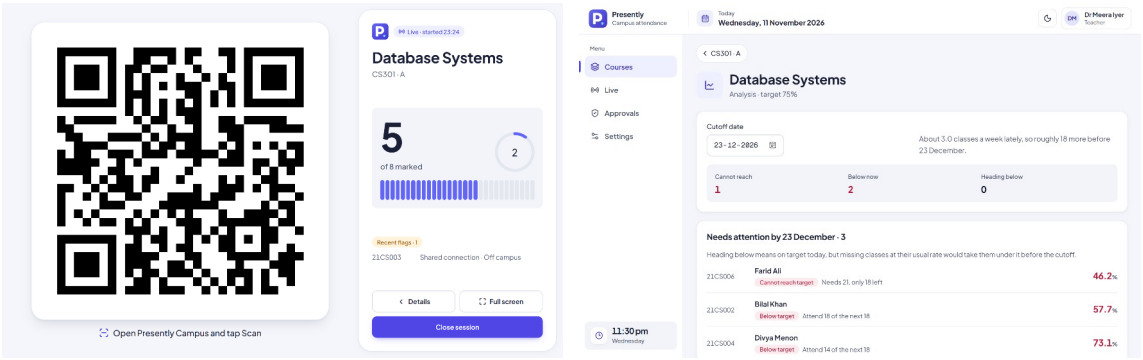


Figure 2: *A course seen by the teacher: summary figures, the roster and the session list*

Starting a session opens the projector view (Figure 3, left): a large rotating code, a ring counting down to the next code, the live count of students marked and any recent flags. From a session, the teacher can mark a student present or excused by hand with a reason, close the session, reopen it within 15 minutes, or cancel it.

7.2 Analysis and data

Because every attempt is stored, the data supports analysis beyond a register. For each course the teacher can see which students cannot reach the target before a chosen cutoff date, which are below it now, and which are on target today but heading below it at their usual rate of absence (Figure 3, right). The analysis view also charts attendance class by class and absence by weekday and hour, and shows how often each kind of flag was confirmed or dismissed. The register, or every mark with its method and flags, can be exported as CSV.

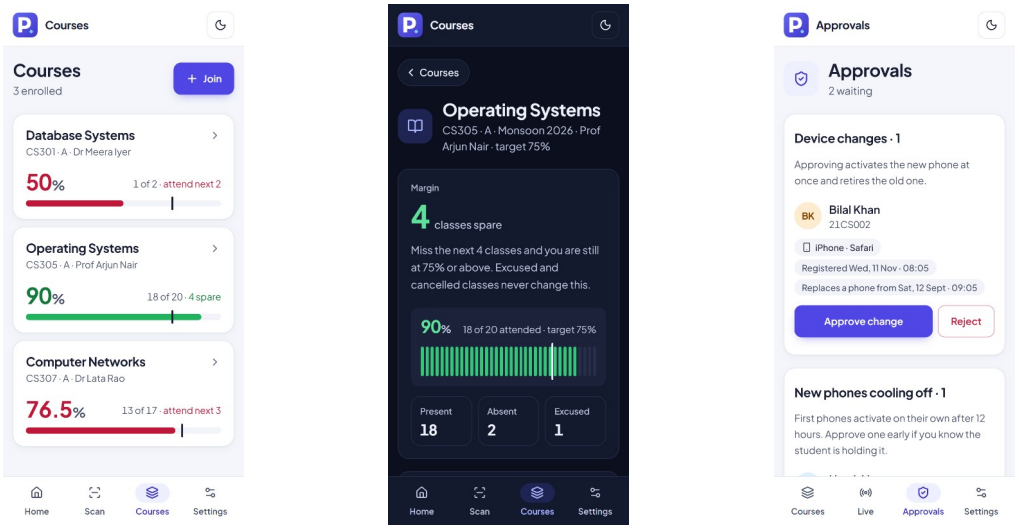


Projector view during a live session — Analysis before a cutoff date

Figure 3: *The teacher's projector and analysis views*

7.3 Student's application

Students have four tabs: Home, Scan, Courses and Settings. Home shows any live session in their courses with a large Scan button, and the course closest to the target. Each course shows the percentage and the margin in plain words, such as "4 classes spare" or "attend the next 2" (Figure 4). Settings shows the phone's registration status and lets the student request a change of phone, which a teacher then approves.



Student: courses — Student: one course (dark) — Teacher: approving a phone

Figure 4: *The application on a phone*

The interface is designed to be used without training by teachers who are not used to phone apps: 17-pixel body text with three text sizes, buttons at least 52 pixels tall, a name beside every icon, light and dark themes with strong contrast, and colour always paired with words (green on target, amber close, red below).

7.4 Development practice

The two members owned separate areas and worked through pull requests on GitHub, with each commit credited to the owner of the area it changed. The `main` branch deploys automatically to Vercel. Every commit had to pass `npm run check`: type checking, all automated tests and a production build.

Table 6: *Division of work*

Member	Areas owned
Nabaskar Brahma	Project setup; database schema, RLS and functions; QR token, phone registration and scan verification; student screens
Abhishek Kokadwar	Teacher screens: courses, live session, session detail, approvals; flags, CSV export, live count and analysis; acceptance testing

8. Testing

8.1 Automated tests

The project has **184 automated tests in 20 files**, run on every commit, and **six SQL test files** run against a real PostgreSQL database in Docker.

- **Token module:** valid code, previous window, expired code, future window, tampered signature, wrong session.
- **Scan and registration rules:** every one of the nine checks and every phone rule, run in Node against an in-memory store.
- **Attendance maths:** percentages and margins, including excused, cancelled and late-joined cases.
- **Database:** role assignment, RLS isolation (a student cannot read another student's attendance; a teacher cannot read another teacher's course), permission checks on every function, flags and export.
- **Screens:** the student and teacher applications driven through real user actions against a fake backend, including the projector's clock correction for a laptop two minutes wrong.

8.2 Acceptance tests

Each anti-proxy acceptance test from the design is reproduced as an automated test that runs the real scan rules and database policies, so every one of them is re-checked on every commit. All of them pass (Table 7).

Table 7: *Acceptance tests and their results*

#	Attempt	Expected and observed result	Result
1	Scan the projected code normally	Marked present, with the course name and time	Passed
2	Photograph the code, wait 30 seconds, scan the photo	"Code expired"	Passed
3	Use student B's account on student A's registered phone	"This phone isn't registered to you"	Passed
4	Clear site data, sign in as B, register the phone, scan the same class	"New phones can't mark this class"	Passed
5	Write to the attendance table directly with a student's token	Refused: permission denied	Passed
6	Replay a valid scan request after the session closes	"This class has closed"	Passed
7	Run the projector on a laptop whose clock is 2 minutes wrong	Codes still accepted after clock correction	Passed

#	Attempt	Expected and observed result	Result
8	One phone tries to mark a second student in the same class	"This phone has already marked someone in this class"	Passed
9	Scan twice, including two scans racing each other	"Already marked present"; only one row	Passed
10	Teacher marks a student by hand	Row records method "manual", the reason and the teacher	Passed

9. Results and Discussion

9.1 Proposal against delivery

Table 8 compares what the proposal planned with what this project delivers.

Table 8: *Proposal next steps and what was delivered*

Planned in the proposal	Delivered in Presently Campus	Status
Automatic marking by QR code or Bluetooth beacon	A rotating QR code scanned from each student's registered phone, with nine server-side checks against proxies. QR was chosen over beacons because it needs no hardware in the room.	Built
Class-wide analysis: students at risk, subjects and slots with most absence	At-risk students highlighted on every roster; who cannot reach or is heading below the target; absence by weekday and hour; attendance class by class	Built
Margin as of the eligibility cutoff date	The teacher picks a cutoff date and the analysis projects each student's standing at that date	Built
On-duty and medical leave	The teacher marks the student excused with a reason; excused classes count on neither side	Built
Two-hour labs counted as two	Each session counts as one class	Future scope
Skip decision and recovery plan for students	Kept: "can miss N" and "attend next N" for every course, from the same formulas	Built
Corrections calendar for students	Replaced by design: students can no longer mark; teachers correct marks per session, and every change is logged	Changed
Pilot: one section of about 60 students, four weeks	Beyond this project: a field study to run with the finished system	Future scope

9.2 Outcome

Presently Campus is complete. Every planned feature is built and deployed at presently-campus.vercel.app on its own Supabase project, with sign-up limited to the college domain. All 184 automated tests, all six database test suites and all ten acceptance tests pass. A scan is a single request to the server, so marking is designed to take about a second; for a class of sixty the limit is how quickly students raise their phones, well within the three-minute goal.

9.3 Limitations

- An absent student's own phone, carried into class and unlocked by a friend, is not prevented. The live count against a head count is the current check.
- Safari may clear a website's storage, and the phone's key with it, after seven days without a visit unless the app is installed to the home screen. The student then needs a phone change, which a teacher approves quickly.
- Students without a smartphone must be marked by hand by the teacher.

- A website can identify a browser but not the physical phone, so two accounts registered in two browsers on one phone 12 hours ahead can both be marked from it; the extra mark shows against the head count.

10. Future Scope

The project is complete as delivered. The following would extend it beyond its scope:

- **Passkeys on every scan:** ask for the phone's fingerprint or face unlock (WebAuthn with user verification) before each scan, so that only the phone's owner can mark. This closes the "friend carries the phone" gap.
- **Location flag:** flag scans whose GPS reading is far from the classroom, storing only the flag, never the coordinates.
- **More than one teacher per course,** for lab assistants and shared courses.
- **Lab weights,** so that a two-hour lab counts as two classes, as planned in the proposal.
- **ERP integration,** to import rosters and send final attendance to the college portal.
- **A semester-long field study** with one section of about sixty students, measuring time to mark a class, first-try scan success, proxies confirmed by the teacher, and how closely the figures match the official register.

11. Conclusion

Our proposal described a personal tracker whose figures depended on students' own taps, and set three next steps. Presently Campus completes them. It replaces roll calls with a scan that takes about a second per student, while making proxy attendance impractical. It does this by combining a QR code that expires in seconds, a cryptographic key that ties each account to one phone, and nine checks on the server that the browser cannot bypass. Students always know how many classes they can miss in each subject, teachers see the whole class at a glance, and every scan attempt becomes data that can be analysed. The system is complete, deployed, and verified by 184 automated tests, six database test suites and ten acceptance tests, all passing.

References

[1] Supabase. *Supabase Documentation: Database, Auth, Edge Functions, Realtime.* https://supabase.com/docs

[2] The PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies.* https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[3] H. Krawczyk, M. Bellare and R. Canetti. *HMAC: Keyed-Hashing for Message Authentication.* RFC 2104, IETF, 1997.

[4] D. M'Raihi, S. Machani, M. Pei and J. Rydell. *TOTP: Time-Based One-Time Password Algorithm.* RFC 6238, IETF, 2011.

[5] National Institute of Standards and Technology. *Digital Signature Standard (DSS).* FIPS 186-5, 2023.

[6] W3C. *Web Cryptography API.* W3C Recommendation. https://www.w3.org/TR/WebCryptoAPI/

[7] W3C. *Web Authentication: An API for accessing Public Key Credentials, Level 2.* W3C Recommendation, 2021.

[8] Nimiq. *qr-scanner: Lightweight JavaScript QR Code Scanner.* https://github.com/nimiq/qr-scanner

[9] Government of India. *The Digital Personal Data Protection Act, 2023.* Ministry of Law and Justice.